

SmartClaim AI: Code and Commands Explanation

1. Explanation of Google Colab Commands

The commands entered in the Colab Notebook are designed to download the project, install its dependencies, and make the local web server available on the public internet so you can view it from anywhere.

```
Command: !git clone https://github.com/ahmadnadeembutt/smartclaim.git
```

This command downloads (clones) the entire SmartClaim repository from GitHub into the Google Colab environment. It brings in all the python scripts, models, and data needed.

```
Command: %cd smartclaim  
!pip install -r requirements.txt
```

The `%cd` command changes the current directory to the newly downloaded folder. The `pip install` command reads the `requirements.txt` file and installs all the necessary Python libraries required to run the AI model and the frontend (like Streamlit, Pandas, Scikit-learn, Sentence-Transformers).

```
Command: !streamlit run app.py &>/content/logs.txt &  
!npx localtunnel --port 8501
```

`!streamlit run app.py`: This starts the Streamlit web server. The `&` at the end tells Colab to run it in the background so the cell doesn't get stuck. The output is saved to `logs.txt`.

`!npx localtunnel`: Streamlit runs locally inside Colab on port 8501. Because Colab is a closed cloud environment, you cannot access port 8501 directly. Localtunnel creates a temporary public URL that securely forwards traffic from the internet to port 8501 inside Colab.

2. Explanation of the Frontend Code (app.py)

The `app.py` file uses Streamlit to create the user interface. It acts as the bridge between the user and the Machine Learning models.

```
A. Model Loading Function:  
@st.cache_resource()  
def load_ml_pipeline(): ...
```

Loading deep learning models takes a lot of time. By using `@st.cache_resource()`, Streamlit ensures that the heavy AI models are loaded into the server's memory only once when the app starts. When a user clicks 'Predict', the model is already in memory, making predictions instantaneous.

B. Page Navigation System

Streamlit uses a sidebar for navigation (`st.sidebar.radio()`). Based on the user's selection, different functions are called to render different pages (e.g., `page_predict()`, `page_methodology()`, `page_about()`). This keeps the app organized into a multi-page dashboard without needing complex HTML/CSS routing.

C. The Prediction Page Logic

The main page contains a Text Area where users enter their Arabic accident description. Below it are 'Quick Example' buttons. When an example button is clicked, it uses a **Callback Function** (`on_click=set_example_text`) to instantly update the text area's internal state variable without breaking Streamlit's rendering cycle.

D. Integration with Backend

```
from src.predict import predict_cost
...
result = predict_cost(user_text)
```

When the user clicks 'Analyze & Predict', the `app.py` file takes the text from the text box and passes it to the `predict_cost()` function imported from the backend (`src/predict.py`). It then takes the resulting predicted cost category (like 'Minor', 'Moderate', or 'Severe') and displays it beautifully using Streamlit metrics and success banners.

E. UI Aesthetics and Styling

The app uses custom CSS injected via `st.markdown(..., unsafe_allow_html=True)` to enforce a premium Dark Theme with custom fonts, glassmorphism effects on cards, and glowing hover states on buttons.